Modelling Hub and Spoke Logistics Networks with Juggling Frameworks
Part B Progress Report

Syed Uzair Shah Bukhari

05/08/2026

Project Supervisor: Anthony Mays

Declaration

In submitting this work I am indicating that I have read the University's Academic Integrity Policy. I declare that all material in this assessment is my own work except where there is clear acknowledgment and reference to the work of others. I give permission for this work to be reproduced and submitted to other academic staff for educational purposes.

1 Abstract

Hub-and-spoke networks are widely used in the transportation industry in order to keep costs low by transporting items from multiple far off locations to a central hub from which they are delivered to a common final destination. However, this might lead to bottlenecks at the hub which has a finite amount of unloading resources. We implement a juggling framework in order to model a system of up to 3 trucks that can travel from the hub to its spokes and view it through a combinatorics lens. This allows for the arrival of trucks to be viewed as juggling states. In particular, we concentrate on the arrival of multiple trucks to the hub which results in trucks having to wait at the hub. This impacts both the supply chain network downstream in terms of delayed arrival of goods and the finances of logistics companies.

From our model it is found that the hub's unloading capacity and the travel times of trucks have an impact on the economics of the hub-and-spoke system. Increasing the number of unloading bays from one to two increases profit, but a third bay yields little benefit as it is rarely utilised. On the other hand longer travel times lower both revenue and cost as fewer trucks are unloaded over time. The model requires validation against a real-world system before it can be used to make any recommendations.

2 Introduction

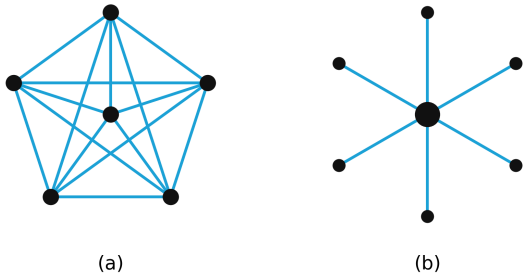


Figure 1: Comparison of hub-and-spoke (HS) network and point to point (P2P) network structure. Source: [16] [15]

Hub and spoke networks are widely used across different industries including the logistics [13, p.1], aviation [14, p.1-2] and cloud computing industry [2]. In a traditional point to point transport (P2P) network (see a in 1), goods (or passengers) are transported from dispatch location to the final destination. However, it may not be feasible in terms of cost to make these deliveries to the final destination. This is because shipping to multiple different final destinations requires dedicated trucks on each route by each dispatch location. This can result in cost loss caused by insufficient trucks with a full load being carried [13, p.1]. An alternative to this P2P network is a hub and spoke (HS) (see b in Figure 1) network where goods are collected at a central hub from which they are shipped to the final destinations which we call spokes [1]. This means that the goods can be collected from various dispatch points and shipped in bulk to their final destinations. This reduces the number of dedicated trips from each dispatch location. However, the hub's unloading capacity is finite and can therefore serve as a bottleneck when multiple trucks arrive simultaneously. Congestion and transport delays at the hub can have detrimental effects downstream on the logistics network resulting in an overload or possible failure of the hub [13, p.2]. This is why trip scheduling is an important tactic to avoid such a scenario.

Trip scheduling has been looked at primarily through the lens of optimisation techniques such as cross entropy optimisation [7] which searches different permutations to minimise span of the schedule, i.e. the total time required to complete all trips. However, such optimisation techniques have not been studied in detail for this paper. We pay attention to combinatorial frameworks to tackle trip scheduling.

Juggling models provide a natural format to frame this problem by implementing combinatorics. Juggling states are able to encode the current and future arrivals of trucks to the hub with the passage of time. The same way a juggler schedules balls to land on future beats, trips are scheduled to arrive at the hub at future points in time. By treating these arrivals as random, the tools of juggling state graphs and Markov chains can be used to study the long-run behaviour of a hub-and-spoke network.

Using this framework, we model a hub served by up-to three trucks and simulate its behaviour under varying number of unloading bays and travel times. Our experiments show that hub capacity and trip timing both have an economic impact. Increasing the number of bays from one to two improves profitability, but adding a third bay yields little benefit as it is rarely utilised. On the other hand increasing travel times reduces both revenue and cost as less number of trucks are unloaded over time. These results illustrate how juggling-based models can inform decisions about hub capacity and trip scheduling, although we note the model has not yet been calibrated against a real hub-spoke network.

3 Background

A basic hub-and-spoke transport network is displayed in Figure 2. Here, the central hub is the circle labelled as Hub and the spokes are ports located at varying distances from the hub labelled A till C. Trucks route their cargo through a central hub from their spokes. The benefit of utilising a hub-and-spoke model has been noted by [14, p.2] in the context of the aviation industry where it serves to maximise profits. This is because airlines are able to carry a number of passengers with varying final destinations to the same central airport.

However, as described by [6, p.1–2] for their train model which is similar to a hub-and-spoke network, the trains may have to queue up at the central port (Hub) when arriving from silos (spokes A-C) at the same time. This may be due to limited unloading capacities at the port. Juggling notation has been used

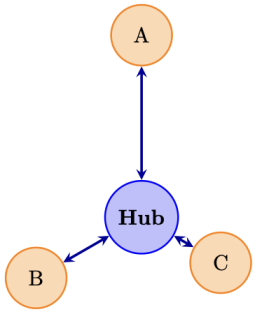


Figure 2: Hub and Spoke Transport Network with three spokes at varying distances. Sources: [17] [19]

to model this train system in [6, p.9]'s talk but they do not continue with this approach in depth in the related paper [7]. Instead, they focus on an optimisation method to construct their trip sequences.

3.1 Siteswap Notation and Juggling Models

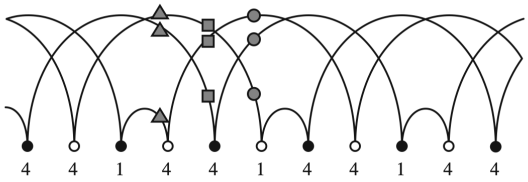


Figure 3: Juggling diagram of juggling pattern 441. Source: [5, p.46]

In order to describe juggling models, a notation was developed by various mathematicians around the same time and has been described by [3, p.5–6] in detail. It is known as siteswap notation. According to this notation, time is counted in the form of discrete beats. It is assumed that a juggler throws and catches balls for an infinite amount of time with no lag between catching and throwing the ball. Therefore, catching and throwing can happen on the same beat. When the juggler throws this ball, it is associated with a certain non-negative integer h which is called "throw-height". As such, this h-throw lasts h beats of time. This can be observed in figure 3 where each black and white circle represents a ball caught at a time beat. The numbers 4, 4 and 1 represent the smallest repeating pattern of throw heights whose length is called its period. For example, a ball thrown at a height of 4 is caught 4 beats later and the period of the simple juggling pattern 441 is 3.

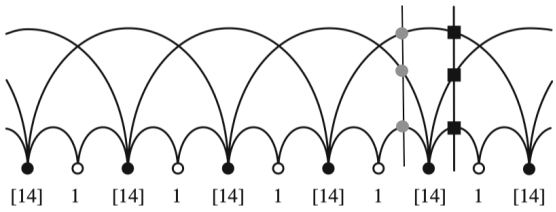


Figure 4: The juggling diagram of the multiplex juggling sequence [14]1. Source: [5, p.66]

Juggling patterns are split into two classes: simple and "multiplex". In a simple pattern, at most one ball can be caught and thrown on any given beat [5, p.8], whereas a multiplex pattern relaxes this restriction by allowing more than one ball to be caught and thrown simultaneously [5, p.65]. Both classes are characterised by two parameters: the number of balls being juggled, b, and the maximum throw-height, h. A simple pattern such as 441 specifies a single throw-height per beat. However, for a multiplex pattern, we use square brackets to group simultaneous throws on the same beat. Note that in figure 4, square brackets are used to show simultaneous throw-height and the juggling pattern is [14]1. Reading from left to right, the first black round circle represents 2 balls thrown at a height of 1 and 4 respectively. The first white circle that follows represents a single ball thrown to a height of 1. This pattern repeats for the remaining figure.

Additionally, we will describe the "add-drop" model as mentioned in [4, p.115]. In this model, a

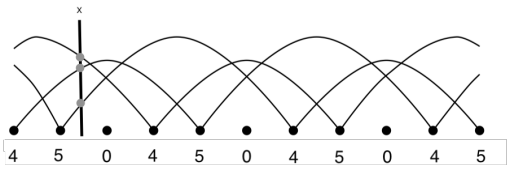


Figure 5: Juggling diagram of a juggling sequence 450 where a ball can be added or dropped. Source: [3, p.9]

juggler can add additional balls to the pattern or drop existing ones. If the juggler faces a wait i.e. no ball landing at t=0 as can be seen after the vertical line in figure 5, the juggler can add an extra ball to the pattern at this beat. If they have a ball at t=0, they can choose to drop this ball as well.

Hub-and-spoke models can be described using juggling models. Juggling describes the arrival of a sequence of balls in a juggler's hands in discretised time after they have been thrown to a certain height. This mirrors the arrival of trucks to a hub at hourly intervals. The trucks can be treated as balls whereas the arrival time of a truck can be treated as the height of the throw of a ball. For example, if a juggler throws a ball with a throw-height of 3, the ball will land after 3 time beats. This is equivalent to a truck returning to the hub in three hours where each beat represents an hour. Lastly, multiplex juggling allows for multiple balls to land in a juggler's hands at the same time. This is analogous to multiple trucks arriving simultaneously at a central hub.

3.2 State Transitions

Based on the notation described above, throw-heights determine the beat at which the ball will land in the future. As described in [5, p.44], these landings can be represented as a pattern of 1's and 0's. Assuming that the juggler has been captured in a moment in time, represented by the three triangles in figure 3. These balls (represented by the triangles) will land in the juggler's hands in successive beats one after the other. The landing state, which is also described as the juggling state, can be written as 11100. Here, each number is separated by a time beat where 1 represents a ball landing and 0 represents no balls landing. Since a ball can be thrown up to 5 beats into the future, extra 0's are included to account for those potential landing slots as shown in figure 6. The ball at t=0 is thrown to a height of 4 thereby reaching state 11010. Note that the remaining 1's and 0's between t=1 till t=4 shift to the left by one time step to indicate that a beat has passed between the transition. The state transition described can be seen in figure 6. The intermediate step is omitted in later transition diagrams.

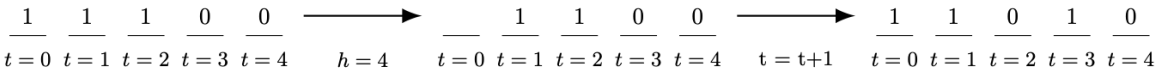


Figure 6: Transition between juggling states for a throw of height $h = 4$. Source: [20] [22]

However, different rules apply to simple and multiplex juggling states. In a simple juggling model, multiple balls cannot be caught (or thrown) at the same time. This means that two balls cannot arrive simultaneously in the future. However, multiplex juggling allows for this constraint to be lifted. Observe the vertical line that intersects the path of the three balls thrown by the juggler with grey circles in figure 4. We note the path of these three balls. The first two balls lands at t=0 and the third ball lands at t=2. This results in a juggling state 20100 where we assume a maximum throw-height of 5. By throwing the balls at t=0 through a height of 1 and 4, we land at the state 11010. This representation can be seen in figure 7

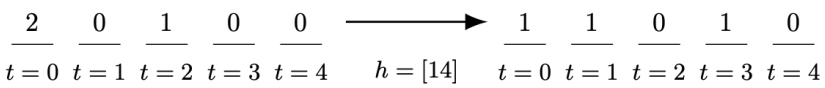


Figure 7: Diagram representing transitions between juggling states and their respective throw-heights for a multiplex juggling system. Source: [20]

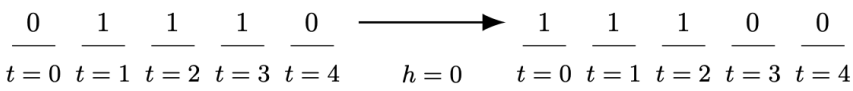


Figure 8: Transition between juggling states where no ball is thrown or caught denoted by throw-height of 0. Source: [20]

Additionally, if the juggler is faced with the situation of no balls arriving at the current time beat such as 01110, no action will be required by the juggler and such a throw is called a 0-throw. The sequence moves to the left resulting in the state 11100 as seen in figure 8. However, this is not the only 0-throw. Adding or dropping a ball is also considered a 0-throw. If a juggler has a ball at t=0, they can drop it and this can be seen by the dashed arrow in figure 9. If a juggler has no ball at t=0, an additional ball can be inserted anywhere in the sequence to increase the number of balls being juggled. This situation can be seen with the dotted arrow in figure 10. Note that, in this system, addition and dropping of balls is treated by moving one time step forward after addition or removal of the balls.

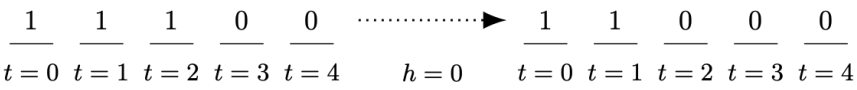


Figure 9: Transition between juggling states where a ball is dropped from the system. Source: [20]

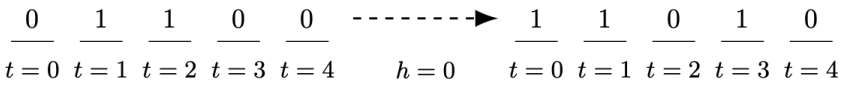


Figure 10: Transition between juggling states where a ball is added to the system. Source: [20]

3.3 Juggling State Graph

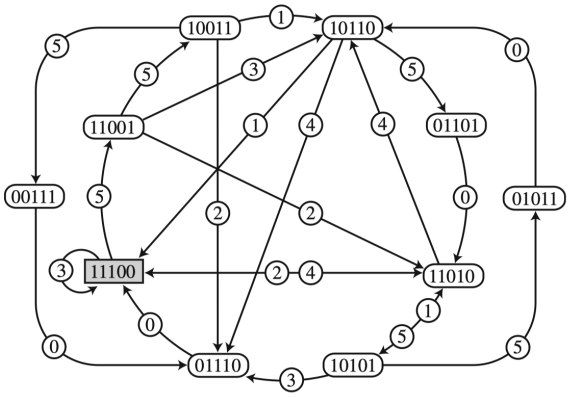


Figure 11: The simple 3 ball juggling state graph of height 5. Source: [5, p.45]

By finding out the total states and the throw-heights that connect them, we can create a graph known as a "juggling state graph". The graph for simple juggling can be seen in figure 11. Here we note that to go from state 11001 to state 10011, we require a throw-height of 5 whereas to reach state 11100 from state 01110, a 0-throw is required.

3.4 Uniform Distribution & Transition Matrix

We note that a juggler has the ability to choose which state to transition into next based on the throw-height. We assume that the juggler is equally likely to choose from n allowed throw-heights while in a given state i in order to reach the next state j. Therefore, the juggler follows a uniform random process [25, p.105] where the probability of transitioning from state i to state j is defined in equation 3.1. We

also assume that the future juggling state only depends on the current juggling state and permissible throw-heights.

$$P_{ij} = \begin{cases} \frac{1}{n} & \text{if state } j \text{ is reachable from state } i, \\ 0 & \text{otherwise} \end{cases} \tag{3.1}$$

Juggling systems can be then be seen as stochastic processes where future juggling states are conditionally independent of their past given the present, that is they obey the Markov property as shown in equation 3.2 [11, p.1715]:

$$P(X_{t+1} = x | X_1 = x_1, X_2 = x_2, ..., X_t = x_t) = P(X_{t+1} = x | X_t = x_t) \tag{3.2}$$

According to [11, p.1715], when the probabilities of transitioning between states are independent of time, where time is measured in discretised steps, the probabilities can be collected into a square matrix P known as the transition matrix, where each entry Pij gives the probability of moving from state i to state j as seen in equation 3.3.

$$P = \begin{pmatrix} P_{11} & P_{12} & P_{13} \\ P_{21} & P_{22} & P_{23} \\ P_{31} & P_{32} & P_{33} \end{pmatrix} \tag{3.3}$$

3.5 Stationary Distribution

Theorem 3.1 (Perron-Frobenius Theorem [10, p.28])**.** *If a Markov chain is connected and aperiodic then*

- *There is a unique stationary distribution* π
- *For every initial vector p(0), we have*

$$lim_{m \to \infty} p(0) P^m = \pi \tag{3.4}$$

Since the possible juggling states and possible transition probabilities between states are known, juggling state graphs can be encoded into a transition matrix P in order to observe the long run behaviour of the juggling system. This approach has been taken by [4, p.110] who had to calculate the stationary distribution vector $\vec{\pi}$ whose entries π_i can be interpreted as the fraction of time spent by the Markov chain in the ith state. This interpretation is valid when the stationary distribution exists and is unique. Since our finite Markov chain is connected and aperiodic as per definitions A.3 and A.4, theorem 3.1 guarantees that the stationary distribution π exists and is unique.

$$(P^T - I)\vec{\pi}^T = 0 \tag{3.5}$$

where I is the identity matrix.

In order to calculate this stationary distribution, [4, p.110] finds the left eigenvector for P corresponding to the eigenvalue 1 (for a definition of eigenvalue and eigenvectors refer here: A.1). The details of how to calculate this have been outlined in [9, p.19–21] and involves solving the homogeneous equation 3.5. We note that the solution is of the form $t\vec{v}$ where $t \in \Re$. This means that there are multiple solutions which are all scalar multiples of each other. In our case, the stationary distribution is a probability vector and therefore requires that $\sum \pi_i = 1$. Therefore, we normalise the solution obtained using equation 3.6 to find the stationary distribution vector.

$$\vec{\pi} = \frac{t\vec{v}}{t \sum v_i} \tag{3.6}$$

From here on, we will apply the juggling framework on the hub-and-spoke network defined above. Trucks travelling between hub and spokes will be modelled as balls, with their total travel time corresponding to the height of throws for the balls in the juggling system and time is counted in hourly beats. In context of the hub-and-spoke model, the stationary distribution of our resulting Markov chain describes the fraction of time the network spends in each state of trucks in transit in the long run.

4 Methods

4.1 Formulating the model and constructing the state graph

In order to describe our model which utilises the juggling framework described in the sections above and applies it to the hub-and-spoke network, we gradually build it up below.

4.1.1 Multiplex juggling applied to hub-and-spoke network

Assume that a company runs a hub-and-spoke model with three trucks. Depending on how far the spoke is from the hub, the travel times will vary. In this model, we assume that there are three spokes as seen in figure 2 where the farthest spoke is three hours away. Time is counted in hourly increments (also known as beats). These trucks are constantly moving and delivering goods between the central hub and the three spokes. There is only one bay to unload trucks in a single beat. This means that if multiple trucks arrive at the hub simultaneously, only one truck can be unloaded on a single beat.

We model this system using juggling sequences, where a truck's travel time back to the hub is represented as the height of a throw. If a truck departs from the hub and takes 3 hours to arrive back at the hub, it is considered a 3-throw. To explain this, we focus on an example. If we start with state 100, thrown to a height of 3, after one hour we would reach state 001 as shown in figure 12 where the juggling states are created from the perspective of the hub.

To understand how multiple trucks arriving simultaneously is dealt with, we look at another example. If two trucks arrive simultaneously, the corresponding state is 200 shown in figure 12. Since there is only one bay, one of these trucks would be thrown in to the future with a 1-throw to reflect that this is delayed and will be unloaded an hour later. Whereas, the other truck will be thrown to heights between 0 till 3. If the two trucks are thrown with a throw-height of 2 and 1 respectively, the throw heights would be represented as [21]. An hour later, the new state would be 110 as can be seen in figure 12.

Lastly, it is assumed that the trucks can be thrown to any permissible height ranging from 0 till 3 to reach state j given that they are in state i and all throw-heights are equally likely. Therefore, we can assume that a uniform probability distribution is followed.

4.1.2 Adding or removing trucks

Trucks can be added or dropped from the system under certain conditions. In the real world, this would be equivalent to adding a truck to cater to additional routes to cater to increased demand or removing a truck because it has broken down or requires refuelling. Trucks can be added at any time step if their addition does not exceed the maximum number of trucks allowed, which is three for our model, and they can only be added if there is no truck present at time t=0. Additionally, adding trucks is considered instantaneous that is time is not moved forward on addition of a truck to the system. The trucks can be dropped if there is at least 1 truck available at time t=0. Adding or dropping a truck is denoted by a throw-height of 0. To differentiate these two in the juggling state graph we denote addition using a dotted arrow while a removal is denoted using a dashed arrow in the juggling state graph 12.

4.1.3 Varying the number of unloading bays

To see the impact of unloading more than one truck in a single beat, we add in up to three bays. This allows us to unload two if there are two bays and three trucks if there are three bays simultaneously. We also assume that the maximum number of trucks that can be added or remove in a single beat depends on the number of bays. So if there are two bays, two trucks can be added or removed from the system in one time step.

4.1.4 Varying the throw-height

Lastly, we vary the throw-height while keeping the number of cars limited to three to see the impact on simultaneous arrivals of trucks to the hub. The aim being that if the throw-height is doubled, then it will take trucks longer to return to the hub. This would reduce the number of simultaneous arrivals to the hub.

4.1.5 Final model

Our final juggling model is denoted $J(\leq b, h, B)$. Here, b denotes the maximum number of trucks that our system can carry, h denotes the maximum throw-height and B denotes the number of bays in the

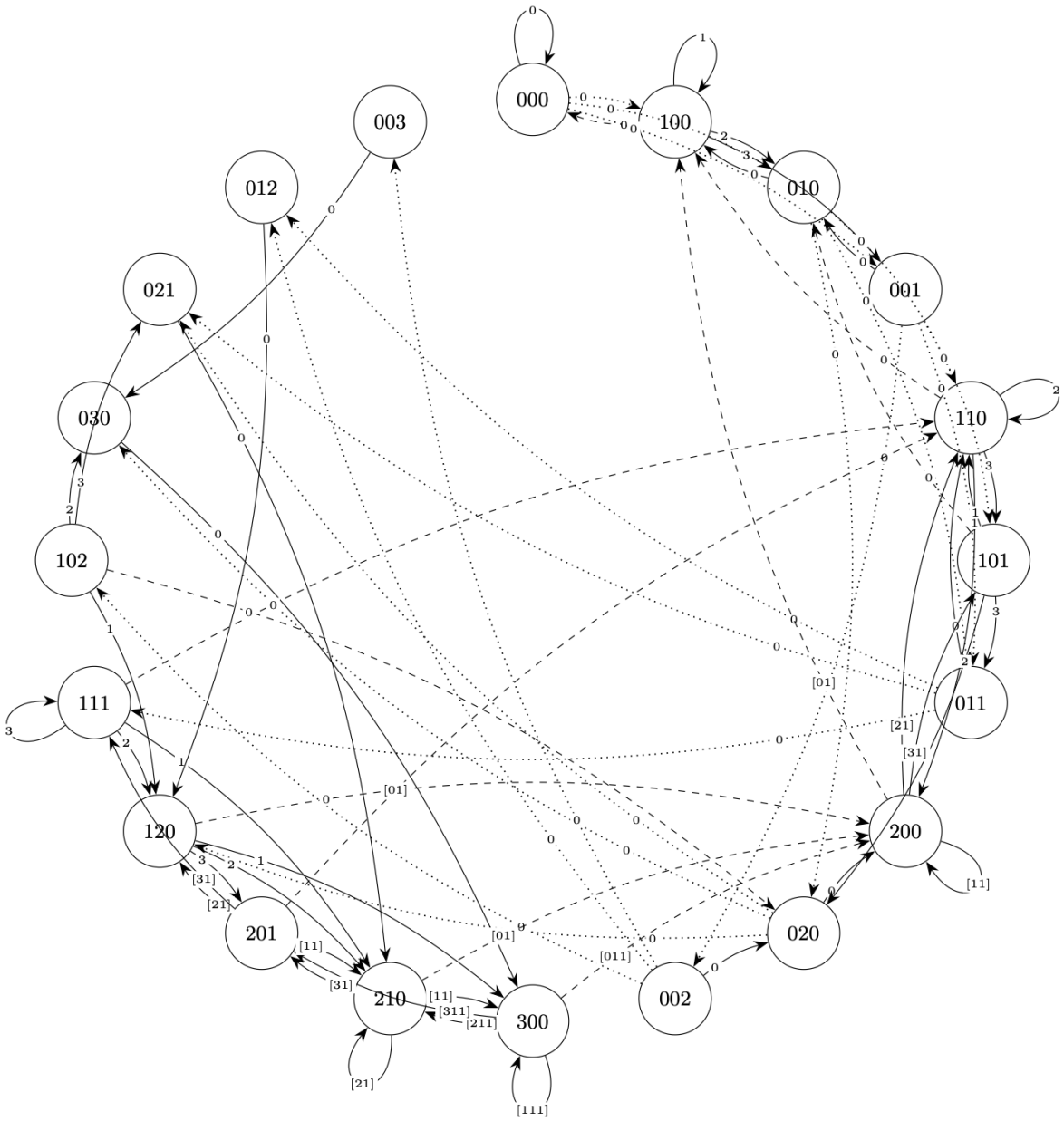


Figure 12: Juggling state graph of up to 3 trucks that can travel for a maximum of 3 hours between the hub with 1 unloading bay and spoke for $J(\leq 3, 3, 1)$. Source: [18]

system. In particular, our primary model $J(\leq 3, 3, 1)$ has one bay. It's juggling state graph can be seen in figure 12. We will later vary the number of bays and throw-heights to see their respective impacts on our system.

We will now move on to finding out how to figure out the total number of valid states there are in this system given that there is only one unloading bay available.

4.2 Number of states

In order to find the number of states for a multiplex juggling system, we use the stars and bars method outlined in section A.2. Since, the number of trucks in our system vary, we add up all the juggling states for the different number of balls. We are able to do so by using equation A.5 where h is the maximum throw-height, b is the number of trucks and b_max is the maximum number of trucks allowed in our system.

However, if we extend the number of trucks and height to values greater than 3, the equation changes. This is because of the limited capacity to unload trucks at the unloading bays preventing certain throw heights leading to illegal states. Finding the equations for these specific cases extends beyond the scope

of this paper.

Our particular juggling system $J(\leq 3, 3, 1)$ has been shown with all its states and transitions in the state graph in figure 12. Every edge carries a known probability as described in section 3.4. This will let us treat the system as a Markov chain which we can then use to find the long run behaviour of the system.

4.3 Generating juggling states

The algorithm used to generate the states and transitions of $J(\leq 3, 3, 1)$ is given in section A.4 [1]

This algorithm is adjusted slightly so that it can be used for the case where the bays are increased to two [2] and three [3]. The difference lies in how many balls can be added and how many balls can be thrown or dropped. If there are 2 bays, then upto two balls can be added and upto two balls can be thrown to specified heights rather than just being throw to height 1. Additionally if the throw-height is doubled, the algorithm stays the same with the addition of initialising the throw-height to double the value [4].

4.4 Encoding state graph as a transition matrix

By using equation 3.1, we encode the state graph for $J(\leq 3, 3, 1)$ in figure 12 as a transition matrix [5] [6] shown in equation A.6 where the probability of moving from state i to state j is assumed to follow a uniform random distribution. For example, by observing figure 12, we note that state 110 can either transition to state 100 by dropping a ball with a throw-height of 0, state 110 with a throw-height of 2, state 101 with a throw-height of 3 or state 200 with a throw-height of 1. It is assumed that each of these states can be reached with equal probability value of 1/4 and correspond to the entries $P_{5,1}$, $P_{5,5}$, $P_{5,8}$ and $P_{5,10}$ [7] respectively.

By using this transition matrix, the stationary vector is found by finding the eigenvector associated with an eigenvalue of 1. This gives us the theoretical stationary distribution required. We also want to observe changes to the system over time by running a simulation.

4.5 Convergence

To ensure that our simulation is behaving correctly, we measure convergence of the simulation to the theoretical stationary distribution when run over a long period. To do so we calculate the residual sum of squares (RSSQ) [8] using equation 4.1 which sums the squared differences between corresponding components of the simulated and theoretical stationary distributions.

$$RSSQ = \sum_{i=1}^{i=20} (\vec{\pi}_i - \vec{\hat{\pi}}_i)^2 \tag{4.1}$$

where $\vec{\pi}$ [9] is the theoretical stationary state vector and $\vec{\hat{\pi}}$ [10] is the simulated stationary state vector at iteration k where k is between 1 and 10^5

RSSQ is calculated at each iteration and the final RSSQ value is found by averaging the values across all iterations.

4.6 Addition of cost and revenue in the model

In order to calculate the total cost and revenue collected from serving trucks at the hub we employ two equations to generate total revenue and total cost. The cost consists of three components as outlined in equation 4.2; namely

[1] Note that the lines of code are from the file `CreateStMatrix_1Bay.m` unless stated otherwise
[2] The code for this can be found in the file `CreateStMatrix_2Bay.m`
[3] The code for this can be found in the file `CreateStMatrix_3Bay.m`
[4] The code for this can be found in the file `CreateStMatrix_1Bay_6H.m`
[5] Note that the following lines of code are from the file `SimRun_1Bay` unless stated otherwise
[6] The matrix can be found on lines 2–21
[7] The row matrix containing these entries can be found on line 6
[8] This calculation is done on line 93
[9] This result and its calculation can be found on line 28
[10] This calculation can be found on line 92

- Fixed cost: cost associated with running the hub which includes paying the staff and the bills.
- Bay cost: cost of unloading a truck given a certain number of bays. If there is more than one unloading bay, the bay cost goes up to incorporate the cost of running this additional bay.
- Delay cost: cost incurred for delaying a car from being serviced. For example, if two trucks arrive at the hub but there is only one unloading bay to service them, then the other car gets delayed by an hour. This incurs a delay cost in the model.

On the other hand, we have revenue which consists of a single component:

- Base revenue: revenue generated by serving trucks at the unloading bay. The greater the number of trucks unloaded, the greater the revenue generated.

$$TC_t = TC_{t-1} + BC \times NP_t + DC \times NT_t + FC \tag{4.2}$$

$$TR_t = TR_{t-1} + BR \times NT_t \tag{4.3}$$

where,

- t is the current time step in the iterations
- TC_t is the total cost at time t
- BC is the bay cost of unloading one or two or three trucks
- DC is the delay cost of one truck
- FC is the fixed cost of operating the hub
- NP_t is the number of trucks processed (unloaded) at time t
- NT_t is the number of trucks delayed at time t
- TR_t is the total revenue accumulated by time t
- BR is the base revenue

4.7 Running Simulation for $J(\leq 3, 3, 1)$

Algorithm 4.1. In order to observe the changes to the system $J(\leq 3, 3, 1)$ over time, a simulation is run based on Markov Chain Monte Carlo simulation [12, p.3].

1. Load the transition matrix P from equation A.6 where each row lists the transition probabilities from the current state i to future state j [11]
2. Theoretical stationary distribution: Find the eigenvector and eigenvalues of transposed P (P^T) and normalise it [12]
3. Initialise simulation parameters[13], economic constants [14], storage [15] and vectors grouping the juggling states [16].
4. For each seed and each starting state k
   (a) Initialise state vector [17], where each position represents a state, to track the number of times a state is visited in n iterations
   (b) Loop over n iterations [18]
      i. Sample the next state from row k of P using inverse transform sampling [29] [19]. The details are mentioned in section A.6

[11] This can be found on lines 2–21
[12] This calculation can be found on lines 27–28 .
[13] These values can be found on lines 31–33
[14] These values can be found on lines 36–42
[15] These vectors can be found on lines 44–45
[16] These vectors can be found on lines 47–49
[17] This vector can be found on line 59
[18] This loop begins at line 65
[19] The calculation is done on lines 61–75

ii. Update total cost and total revenue using equations 4.2 and 4.3. [20].

iii. Find the simulated stationary vector $\vec{\hat{\pi}}$ using equation 4.4[21]

$$\vec{\hat{\pi}} = \frac{\vec{s}}{\sum_i \vec{s_i}} \tag{4.4}$$

where s is `s_vec` and record RSSQ [22] (equation 4.1)

(c) Store the final simulated stationary vector after n iterations for the kth starting state [23].

5. Compute average cost and average revenue as `(total_cost/num_states)/seeds` and `(total_revenue/num_states)/seeds` [24]

6. Average across all seeds and starting states: the simulated stationary distribution [25] and the RSSQ values, then compute the overall RSSQ [26].

4.8 Experimentation

The algorithm can be extended to different scenarios with a varying number of bays and throw-heights to see which results in a more profitable run of the hub-and-spoke system. We set up four different scenarios listed in table 1.

Our initial hub-and-spoke model operates with one unloading bay and a maximum throw-height of 3. Since there is only one bay, we eventually run into a problem of multiple trucks arriving at the hub simultaneously. To serve multiple trucks at once, we increase the number of bays that serve the trucks in experiment two [27] and three [28]. Constructing and operating this additional bay increases the cost of serving each truck. This is indicated by the increased bay cost in experiment two and three compared to experiment one. Finally, we double the throw-height in experiment four while keeping the number of bays constant to reduce the arrival of multiple trucks to the hub, thereby decreasing the total delay cost. By varying the number of bays and throw-height in our system, we can see if the net profit (equation 4.5) can be increased.

$$AP = AR - AC \tag{4.5}$$

Where,

- AR is the average revenue, averaged across all seeds and different starting states.
- AC is the average cost, averaged across all seeds and different starting states.

5 Results

5.1 Convergence

Figure 13 [29][30] shows mean residual sum of squares (RSSQ) [31] plotted against the number of iterations on a log-log plot for our juggling system $J(\leq 3, 3, 1)$. The RSSQ values have been averaged over different starting states and different seeds. The simulation continues for 10^5 [32] iterations and the RSSQ reduces over time in a linear fashion on the log-log scale. The final mean RSSQ value at the end of the simulation is 3.37×10^{-8} [33] as seen in figure 24 [34] which indicates that the simulated value gets closer to the

[20] These calculations are done on lines 76–87
[21] This calculation is done on line 92
[22] This calculation is done on line 93
[23] This result is stored on line 95
[24] These calculations are done on lines 102–103
[25] This vector is stored on line 107
[26] This value is calculated on line 109. The vector used to calculate this value is calculated on line 107 of the same file
[27] The calculations for this experiment are found in the file `SimRun_2Bay.m`
[28] The calculations for this experiment are found in the file `SimRun_3Bay.m`
[29] The following calculations are from the file `SimRun_1Bay.m` unless stated
[30] This graph has been plotted using the lines 135–144
[31] This calculation is done on line 108
[32] This value is stored on line 33
[33] This calculation can be found on line 109
[34] The calculations in this figure have been done using the lines 98–111

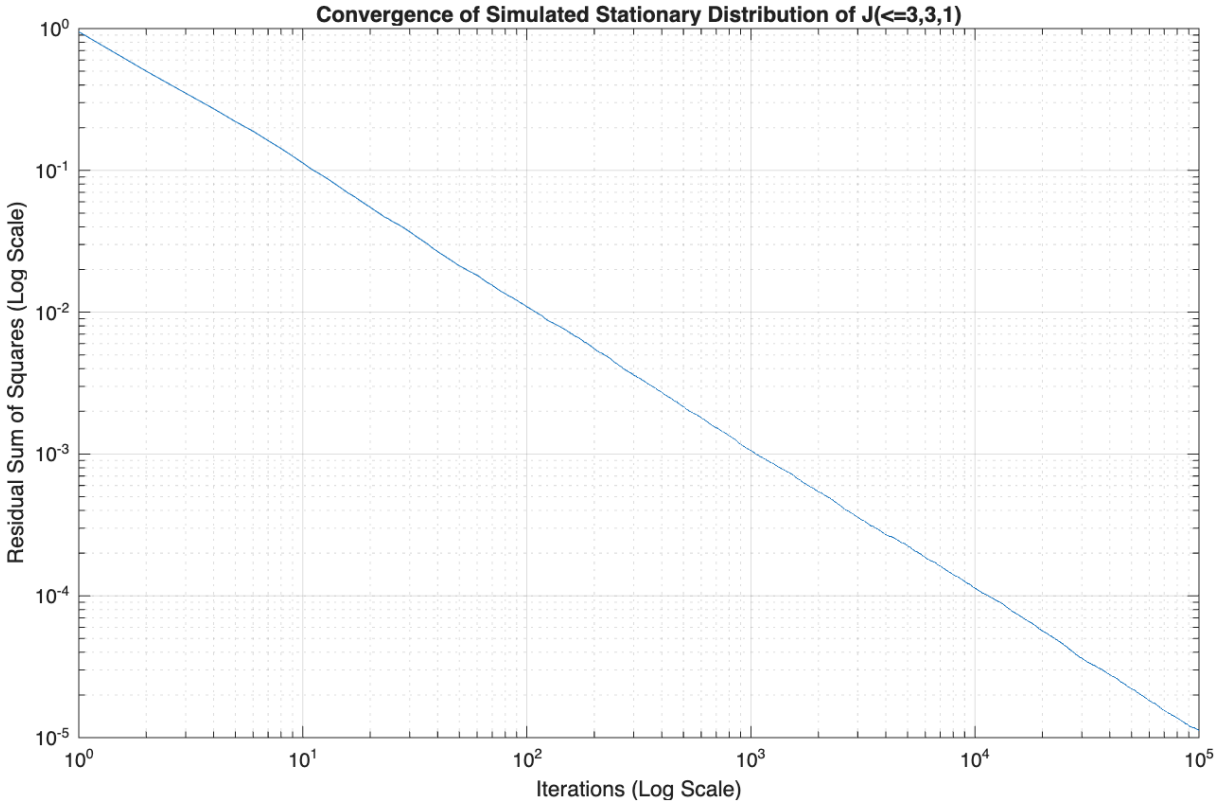


Figure 13: Residual sum of squares plotted against number of iterations to show convergence of simulated stationary distribution with theoretical stationary state for juggling system $J(\leq 3, 3, 1)$ plotted on a log-log plot

theoretical value. Also note that the probability values for the theoretical stationary state and simulated stationary state distributions are nearly identical for all states as can be seen in figure 18 [35]. This trend pertaining to a decreasing RSSQ value has been seen across the different experiments. The RSSQ log-log plots associated with these experiments can be found here: for experiment 2 refer to figure 17 [36], for experiment 3 refer to figure 20 [37] and for experiment 4 refer to figure 22 [38].

5.2 Comparison of Simulated Stationary Distributions

The simulated distribution results for the experiments described above are displayed in figure 14 [39]. These probabilities indicate the fraction of time spent in each state where the states are grouped by the number of trucks arriving at the hub at t=0. We note that the probability of states starting with three is the lowest across all experiments. Secondly, the probability of states beginning with zero increase from experiments one till four with four having the highest probability. Similarly, the probability of states starting with two decreases across the experiments. Lastly, the probability of states beginning with three is the lowest in experiment four. The corresponding un-grouped probability results for experiments one till four can be found in the output images here: 24[40], 25[41], 26[42] and 28[43].

5.3 Economic results

Figure 29 shows the revenue generated in orange and the cost incurred in blue from running the various experiments. Note that the highest cost and highest revenue is generated from experiment three whereas the lowest cost and lowest revenue is generated from experiment four. Figure 30 shows the profit (or loss) incurred from the experiments. Note that the highest profit was approximately 0.20 million which

[35] The graph has been plotted using the lines 147–162 of the file `SimRun_1Bay.m`
[36] This graph has been plotted using the lines 136–145 of the file `SimRun_2Bay.m`
[37] This graph has been plotted using the lines 133–142 of the file `SimRun_2Bay.m`
[38] This graph has been plotted using the lines 198–207 of the file `SimRun_1Bay_6H.m`
[39] This figure is generated using excel and can found in the sheet summarised values in the file `sum_states_2.xlsx`
[40] This terminal has been produced using calculations on the lines 98–118
[41] This terminal has been produced using calculations on the lines 102–121 of the file `SimRun_2Bay`
[42] This terminal has produced using calculations on the lines 99–118 of the file `SimRun_3Bay`
[43] This terminal has produced using calculations on the lines 179–182 of the file `SimRun_1Bay_6H`

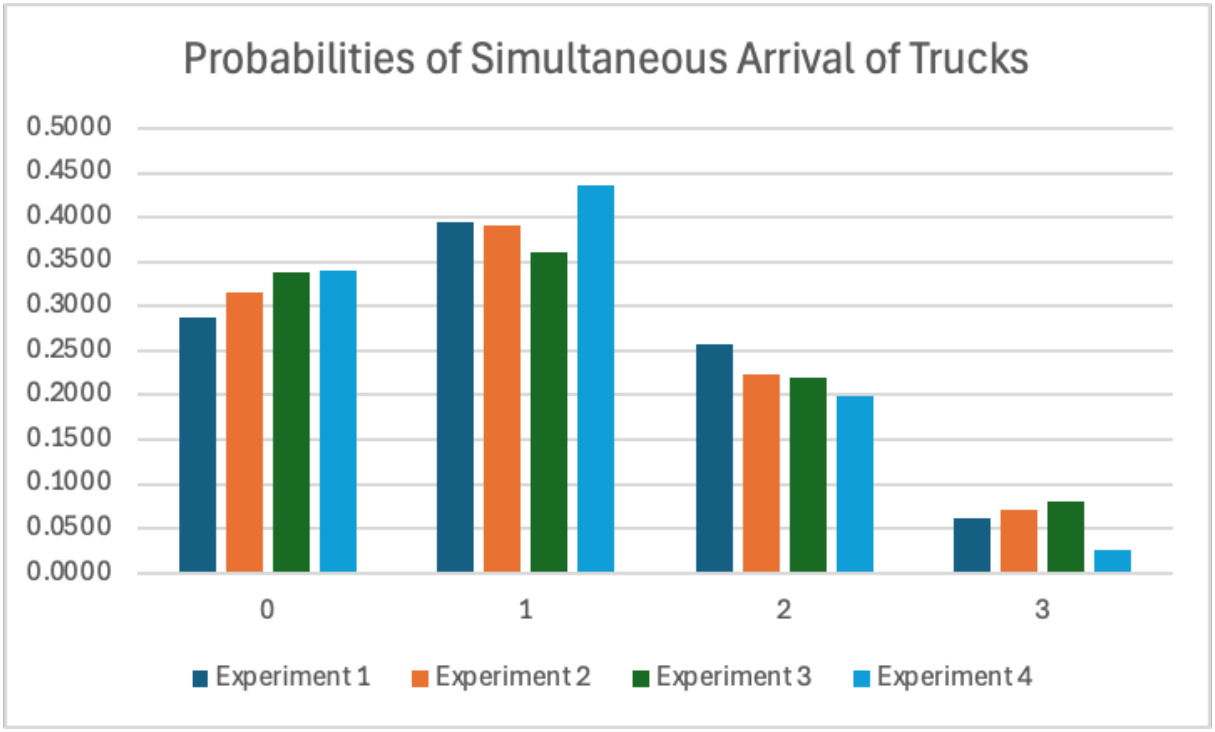


Figure 14: Probability of k trucks arriving simultaneously at the hub for $k \in \{0, 1, 2, 3\}$, under each of the four experiments. The probabilities are calculated by summing the probabilities of each state starting with k for each experiment.

was generated from experiment two. Whereas the highest loss incurred was from experiment one with a value of approximately 0.16 million.

6 Discussion

6.1 Convergence

The RSSQ values approaches zero showing that the simulation methodology has been correctly implemented. Further note that the plots pertaining to experiments one (18 [44]), two (19 [45]), three (21[46]) and four (23[47]) show that in all instances, the simulated stationary state probabilities closely match up with the theoretical stationary state probabilities implying that the simulation behaves correctly.

6.2 Economic analysis

Note that experiment one where only one bay operates has a high cost with low revenue associated with it as seen in figure 29. This results in the largest loss incurred out of all experiments as seen in figure 30. We believe that this is because the number of states with two trucks arriving simultaneously is high when compared to other experiments as can be seen in figure 14. This contributes to a higher total delay cost. We address this using two different approaches as follows:

6.2.1 Increasing the number of bays

In experiment two and experiment three, we increase the number of bays to two and three respectively. This results in a greater revenue in both scenarios compared to experiment one as seen in figure 29. However, experiment two results in a higher profit compared to experiment three as shown in figure 30. We believe this is because in both experiments, the system does not spend a significant proportion of time in states where three cars arrive simultaneously at the hub as seen in figure 14. This leaves the third bay largely unutilised but the additional cost of the third bay in experiment three is still included in the

[44] This graph is plotted using lines 147–162
[45] This graph is plotted using lines 148–163 of the file `SimRun_2Bay.m`
[46] This graph is plotted using lines 145–160 of the file `SimRun_3Bay.m`
[47] This graph is plotted using lines 210–225 of the file `SimRun_1Bay_6H.m`

bay cost for all trucks. This means that the total revenue generated has not significantly increased when compared to experiment two but the total cost does increase greatly as can be seen in figure 29.

6.2.2 Increasing the throw-height

In experiment four, we keep the number of bays constant and double the throw-height. From figure 30, we note a lower loss when compared to experiment one. However, both the revenue and the cost as seen in figure 29 is lower when compared to experiment one. This could be because the system may be spending more time in states where no truck arrives that is states starting with zero as seen in figure 14. Since there is a fixed cost associated with operating the hub, the cost remains high and no revenue is generated when no trucks arrive at the hub.

6.3 Limitations

The primary limitation of this paper is that the model has not been calibrated or validated against any real hub-and-spoke system. We have no evidence that a true network operates as this model does, and significant further work would be required before any real-world recommendations could follow. For now it serves only as a tool for examining how throw-height and the number of bays affect the economics of the system, not as a description of any physical network.

Two structural simplifications widen the gap to a real system. First, the model contains only three trucks and at most three bays, chosen so that every state and transition could be enumerated explicitly; this is far below the traffic an operational hub would face. Second, the assumption that trucks arrive under a uniform random distribution is unlikely to hold in practice, since trucks are dispatched by deliberate scheduling decisions rather than sent at random.

6.4 Future work

Future work would involve addressing the limitations mentioned above with the eventual aim of producing actionable recommendations for operating a hub-and-spoke system. The most important step would be to test the model at a larger scale with increased number of trucks and longer travel times. Secondly, the modelling assumptions would be relaxed; replacing the uniform distribution with a distribution that can better suit scheduled dispatching and actual costs associated with delaying and installing additional bays. The main aim of this calibration would be to closely align the model against a real world system and then use the cost and revenue equations mentioned in the paper to recommend an optimal number of bays and throw-height for the real world system.

6.5 Conclusion

This paper used a Markov chain to model the arrival of trucks to a hub-and-spoke system and used its stationary distribution to analyse the behaviour of the system in the long run. The aim of the paper was to maximise the average profit generated by adjusting the number of bays and throw-height. We found that increasing the number of bays improved the profit generated up to a point: a second bay substantially reduced delay costs, but a third produced little further gain because simultaneous arrivals of three or more trucks were infrequent. On the other hand, doubling the throw-height reduced the frequency of simultaneous arrivals but not enough to make the single-bay configuration profitable, as the fixed costs incurred during the added idle time outweighed the delay savings. These findings hold within a small model and cannot necessarily be generalised to a larger model.

7 Acknowledgements

- I acknowledge that some of this template was adapted from an old University of Adelaide Latex report template.
- Help was provided by an AI agent (Claude.ai) to construct images, create aspects of the code mentioned in section A.4 and help with sentence construction.
  - I acknowledge using my friend's (Shekhar's) AI agent (Claude) subscription.
- Help was provided by my supervisor to develop the ideas of mapping hub-and-spoke systems to multiplex juggling.
- Please find the github link here: https://github.com/uzair-shah/Research-Report

References

[1] Hapag-Lloyd AG. (2024, November 6). The Hub & Spoke System in Shipping and Logistics Explained. Hapag-Lloyd.com; Hapag-Lloyd AG. https://www.hapag-lloyd.com/en/online-business/digital-insights-dock/insights/2024/10/understanding-the-hub-spoke-system-in-container-shipping-.html

[2] Moreno, J., Palacios, A., & Torkar, A. (2025). Hub-spoke network topology in Azure - Azure Architecture Center. In Microsoft.com. https://learn.microsoft.com/en-us/azure/architecture/networking/architecture/hub-spoke

[3] Mays, A. (2006). *Combinatorial aspects of juggling* [Honours Thesis, University of Melbourne]. https://researchers.ms.unimelb.edu.au/~awmays@unimelb/Mays_Juggling_Thesis.pdf

[4] Warrington, G. S. (2005). Juggling Probabilities. The American Mathematical Monthly, 112(2), 105–118. https://doi.org/10.2307/30037409

[5] Polster, B. M. (2003). The Mathematics of Juggling. (1 ed.) Springer-Verlag London Ltd.

[6] Wadhawan, I. B., Pudney, P., Howlett, P., & Piantadosi, J. (2010). *Juggling Trains*. [Slides]. University of South Australia. https://uao365-my.sharepoint.com/:b:/r/personal/a1231659_adelaide_edu_au/Documents/AAStuff/Lecturing/MDSProjects/Juggling_2026/AllPapers/Wadhawan2010_JugglingTrains.pdf?csf=1&web=1&e=SQ7TID

[7] Wadhawan, I. B., Pudney, P., Howlett, P., & Piantadosi, J. (2010). Scheduling trains with cross entropy optimisation. *ANZIAM Journal, 51*, 332. https://doi.org/10.21914/anziamj.v51i0.2594

[8] Math Is Fun. (2025). Invariant (Illustrated Math Dictionary). Math Is Fun. https://www.mathsisfun.com/definitions/invariant.html

[9] Mays, A. (2025). Week 12 Lecture [Lecture Slides]. Adelaide University MyLearning. https://uao365-my.sharepoint.com/:b:/r/personal/a1231659_adelaide_edu_au/Documents/AAStuff/Lecturing/MDSProjects/Juggling_2026/MDS2026_Juggling_Shared/PS_2025_Week12Lecture.pdf?csf=1&web=1&e=JfU6qC

[10] Tuke, J. (2020). Chapter 1: Probability and statistics 2 [Lecture Notes]. Adelaide University MyLearning. https://uao365-my.sharepoint.com/:b:/r/personal/a1947793_adelaide_edu_au/Documents/JugglingProject_Bukhari_SyedUzairShah_a1947793/Books%20and%20Papers/PS_AllNotes.pdf?csf=1&web=1&e=tR0cHB

[11] Seabrook, E., & Wiskott, L. (2023). A Tutorial on the Spectral Theory of Markov Chains. Neural Computation, 35(11), 1713–1796. https://doi.org/10.1162/neco_a_01611

[12] Roughgarden, T., & Valiant, G. (2023). CS168: The Modern Algorithmic Toolbox Lecture #14: Markov Chain Monte Carlo. https://web.stanford.edu/class/cs168/l/l14.pdf

[13] Xu, W., Huang, J., & Qiu, Y. (2021). Study on the Optimization of Hub-and-Spoke Logistics Network regarding Traffic Congestion. Journal of Advanced Transportation, 2021(1), 1–16. https://doi.org/10.1155/2021/8711964

[14] Pels, E. (2020). Optimality of the hub-spoke system: A review of the literature, and directions for future research. Transport Policy, 104, A1–A10. https://doi.org/10.1016/j.tranpol.2020.08.002

[15] Cmglee. (2025). File:Comparison of point to point vs hub and spoke.svg - Wikimedia Commons. In Wikimedia.org. https://commons.wikimedia.org/wiki/file:comparison_of_point_to_point_vs_hub_and_spoke.svg

[16] Anthropic. (2026, July 11th). Side by side image comparison. [Generative AI chat]. Claude. https://claude.ai/share/19ab5fad-5a5b-4420-a59a-9b17127331c3

[17] Anthropic. (2026, July 11th). Three-truck hub and spoke distribution network. [Generative AI chat]. Claude. https://claude.ai/share/cf79a5fa-9cdc-45ff-ad36-68dc71ed997a

[18] Anthropic. (2026, July 20th). Three-truck hub and spoke distribution network. [Generative AI chat]. Claude. https://claude.ai/share/b1a1aee1-e794-4766-9ef8-6014a781fb0d

[19] Anthropic. (2026, July 21st). Hub and spoke network diagram in TikZ. [Generative AI chat]. Claude. https://claude.ai/share/fb9187d9-62e7-4eb3-929e-6503a7750130

[20] Anthropic. (2026, July 21st). Binary sequence diagram with Hamming distances. [Generative AI chat]. Claude.https://claude.ai/share/c5ca28ca-9881-461b-ae1e-03fb42434cbf

[21] Anthropic. (2026, July 22nd). String configuration with TikZ diagrams. [Generative AI chat]. Claude. https://claude.ai/share/fa02ce3c-47c5-4923-a905-f9426a7b07a9

[22] Anthropic. (2026, August 3rd). Clarifying intermediate steps in diagrams. [Generative AI chat]. Claude. https://claude.ai/share/fb5b810e-6e37-4331-a55f-0b58108a99fe

[23] Anthropic. (2026, June 21st). TikZ diagram with dashes and filled circles. [Generative AI chat]. Claude. https://claude.ai/share/3f648ad4-eacd-420c-96ab-4918934b9aa1

[24] Wikipedia Contributors. (2023). Stars and bars (combinatorics). In Wikipedia. https://en.wikipedia.org/wiki/Stars_and_bars_(combinatorics)

[25] Tsun, A. (n.d.). Probability & statistics with applications to computing. Retrieved July 23, 2026, from https://www.alextsun.com/files/Prob_Stat_for_CS_Book.pdf

[26] The MathWorks, Inc. (2022). Matrix of Random Numbers. Mathworks.com. https://www.mathworks.com/help/matlab/ref/double.rand.html

[27] The MathWorks, Inc. (2024). Cumulative Sum of Vector. Mathworks.com. https://www.mathworks.com/help/matlab/ref/double.cumsum.html

[28] Pishro-Nik, H. (2020). Cumulative Distribution Function. Probabilitycourse.com. https://www.probabilitycourse.com/chapter3/3_2_1_cdf.php

[29] Wikipedia. (2023). Inverse transform sampling. In Wikipedia. https://en.wikipedia.org/wiki/Inverse_transform_sampling

Appendices

A Appendix

A.1 Eigenvectors and eigenvalues

This section is referred to in section 3.5.

In order to understand how a stationary distribution is calculated theoretically, we must first define what an eigenvector and eigenvalue is. As per [11, p.1723], an eigenvector of a matrix P is a vector that is only multiplied by some number λ when it is multipled by P. These vectors could be row vectors as in equation A.1

$$\pi^T P = \lambda \pi^T \tag{A.1}$$

in which case they are called left eigenvectors or they could be column vectors as in equation A.2

$$P\pi = \lambda \pi \tag{A.2}$$

in which case they would be called right eigenvectors. In both cases, the number λ is called the eigenvalue of its respective eigenvector.

A.2 Number of states

The following section is referred to in section 4.2.

As per [5, p.77], to find the total number of states for a multiplex juggling system, we can use equation A.3.

$$state_count = \binom{b+h-1}{b} \tag{A.3}$$

where b is the number of balls and h is the maximum throw-height in the system. The method used to come to this formula is known as the stars and bars method and has been explained in [10, p.12–13]. We use a short example with 2 balls and a maximum throw-height of three to explain this method. Since the throw-height is three, we can visualise three time steps separated by vertical lines as shown below.

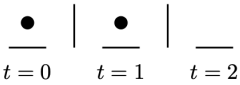


Figure 15: Diagram for representing two balls thrown to a maximum height of 3 at three beats. Source: [23]

If we represent the balls as stars, then we have two balls which we can place on any time step, which are separated by bars. This results in six distinct cases laid out in in figure 16. By using (16), we can find the toal number of ways to do it as shown in equation (A.4).

$$state_count = \binom{2+3-1}{2} = 6 \tag{A.4}$$

If we allow the case where balls can be added or discarded from the system, we can extend equation A.3 by including a summation symbol to calculate all possible juggling states for the multiplex system with a varying number of balls from 0 balls all the way to b_max balls where b_max is the maximum number of balls allowed to be added in to the system as seen in equation A.5

$$total_states_count = \sum_{b=0}^{b_max} \binom{b+h-1}{b} \tag{A.5}$$

T = 0	T = 1	T = 2	String
2	0	0	★ ★ | |
1	1	0	★ | ★ |
1	0	1	★ | | ★
0	2	0	| ★ ★ |
0	1	1	| ★ | ★
0	0	2	| | ★ ★

Figure 16: Six configurations of two balls in three time slots and their stars and bars representations. Sources: [24] [21]

A.3 Transition Matrix

Transition matrix pertaining to our initial model has been written down below and has been referenced in section 4.4 and 4.7

$$P = \frac{1}{4}\begin{bmatrix}
1&1&1&1&0&0&0&0&0&0&0&0&0&0&0&0&0&0&0&0\\
1&1&1&1&0&0&0&0&0&0&0&0&0&0&0&0&0&0&0&0\\
1&0&0&0&1&1&1&0&0&0&0&0&0&0&0&0&0&0&0&0\\
0&0&1&0&0&0&1&1&1&0&0&0&0&0&0&0&0&0&0&0\\
1&0&0&0&1&0&0&1&0&1&0&0&0&0&0&0&0&0&0&0\\
0&0&0&0&0&0&0&0&0&1&1&1&1&0&0&0&0&0&0&0\\
0&0&0&0&1&0&0&0&0&0&0&0&1&1&1&0&0&0&0&0\\
0&0&1&0&1&1&1&0&0&0&0&0&0&0&0&0&0&0&0&0\\
0&0&0&0&0&1&0&0&0&0&0&0&0&0&1&1&1&0&0&0\\
1&0&0&0&1&0&0&1&0&1&0&0&0&0&0&0&0&0&0&0\\
0&0&0&0&0&0&0&0&0&1&0&0&0&0&0&0&0&1&1&1\\
0&0&0&0&0&0&0&0&0&0&0&0&0&0&0&0&0&4&0&0\\
0&0&0&0&0&0&0&0&0&0&0&0&0&0&0&0&0&4&0&0\\
0&0&0&0&1&0&0&0&0&0&1&0&0&1&0&0&0&0&1&0\\
0&0&0&0&0&0&0&0&0&0&4&0&0&0&0&0&0&0&0&0\\
0&0&0&0&0&1&0&0&0&0&1&1&1&0&0&0&0&0&0&0\\
0&0&0&0&0&0&0&0&0&0&0&4&0&0&0&0&0&0&0&0\\
0&0&0&0&0&0&0&0&0&1&0&0&0&0&0&0&0&1&1&1\\
0&0&0&0&0&0&0&0&0&1&0&0&0&0&0&0&0&1&1&1\\
0&0&0&0&1&0&0&0&0&0&1&0&0&1&0&0&0&0&1&0
\end{bmatrix} \tag{A.6}$$

A.4 Generating juggling states and throw-heights to reach them

This section has been referred to in section 4.3

Algorithm A.1. In order to generate the juggling states of our system $J(\leq 3, 3, 1)$ and the throw heights to transition between them, we have created an algorithm to help us with this [48].

1. Setup: Initialise the required variables [49]:
2. Run the algorithm until the queue is empty [50]
   (a) Create an array containing the shifted state which is a subset of the current state from $t = 1$ till $t = end$ [51]
   (b) Create a cell array to capture states and their throw heights called `succs` [52]

[48] Note that the following lines of code are from the file `CreateStMatrix_1Bay.m` unless stated otherwise
[49] The following variables, matrices and arrays are stored on lines 2–7
[50] The loop begins on line 9
[51] This array can be found on line 14
[52] This array can be found on line 15

(c) If there is no ball at t=0, then we can add balls to the current state as long as adding doesn't exceed the maximum number of balls allowed. We then store the new state created with its throw height (0 in this case) in `succs` [53]

(d) If there is one ball at t=0, then this ball can either be dropped or thrown to a certain height (less than maximum allowed height). Different throw heights create different states which are store along with their throw height in `succs`. A ball drop is captured with a throw height of 0. [54]

(e) If there are two balls at t=0, then one ball is thrown (or dropped) as in the previous step, while the other is thrown one time step ahead as there is only one bay to unload the balls. The throw height is captured as [h,1] where h represents throw height and 1 represents the ball thrown one time step ahead [55]

(f) The above is extended to the case where there are three balls with both the remaining balls thrown to a height of 1 each [56]

3. The states are then collected in `states_mat` [57]
4. Location for state i in `states_mat` is collected along with their corresponding future state j's location. We also collect the throw heights required to reach them. This is all added in a single row in `edges` [58]. If duplicate states are reached using a different permutation of throw heights, only one permutation of throw height is recorded in the `edges`
5. The throw heights are collected and placed in a cell array called `transition_values` [59] where their row number coincides with the state they transitioned from and column number coincides with state location they transition to. Additionally, if there is no transitions between the states, the location is left empty.

A.5 Definitions

The following section is referred to in section 3.5.

Definition A.2. (Homogeneous equation) A homogeneous equation has the form

$$Ax = 0 \tag{A.7}$$

where A is a $K \times L$ matrix of coefficients, x is an $L \times 1$ vector of unkowns and 0 is the $K \times 1$ zero vector

Definition A.3. (Connected) In an undirected graph, if there exists a path between each pair of vertices, then the graph is said to be connected [11, p.1746]

Definition A.4. (Aperiodic) When all communicating classes in a set S are aperiodic, the Markov chain is said to be aperiodic.

Definition A.5. (Periodic chains) For each state $s_i \in S$, the period is defined as

$$d_i = gcd\{k \geq 1 : (P^k)_{ii} \geq 0\} \tag{A.8}$$

where gcd indicates the greatest common divisor. Then, equation A.8 says that when starting in state s_i, it is possible to return to s_i only in multiples of the period d_i. States for which $d_i \geq 1$ are called periodic and those for which $d_i = 1$ are called aperiodic.

A.6 Details of the inverse transformation method

Please find the details of the inverse transform method detailed below. This has been mentioned in step 4bi of the algorithm defined in section 4.7

1. Starting from the kth row of P, find the index position of all non zero probabilities and record the probabilities in a vector called `probs` [60]
2. These probabilities are summed up to find the cumulative distribution function CDF [28] [61].which

[53] These calculations are done on lines 18–24
[54] These calculation are done on lines 25–33
[55] These calculations are done on lines 35–43
[56] These calculations can be seen on lines 45–52
[57] This addition is done on line 60
[58] This calculation is done on line 64
[59] These calculations are done on lines 70–72
[60] This vector can be found on line 67
[61] This calculation is done line 70

are stored in a vector called `cum_probs` [62]

3. A random number is generated [63] [26]. The next juggling state is determined by locating which interval of the CDF the random number falls into: if it lies between $F(i-1)$ and $F(i)$, the chain transitions to the ith state [64].

A.7 Experiment Table

This has been referenced here in section 4.8.

Find the experimentation table for $J(\leq h, 3, B)$ where height and bays are varied.

Exp	Bay Count	Bay Cost	Delay Cost	Base Cost	Revenue	Throw Height
1	1	5	5	5	15	3
2	2	7.5	5	5	15	3
3	3	10	5	5	15	3
4	1	5	5	5	15	6

Table 1: Simulation experiments for $J(\leq h, 3, B)$, varying the number of bays B and throw height h.

A.8 Simulation: Plotting results

The following graph 17 has been referred to in section 5.1. This graph describes the residual sum of squares plotted against the number of iterations to show converge of simulated and theoretical stationary distributions for the system $J(\leq 3, 3, 2)$.

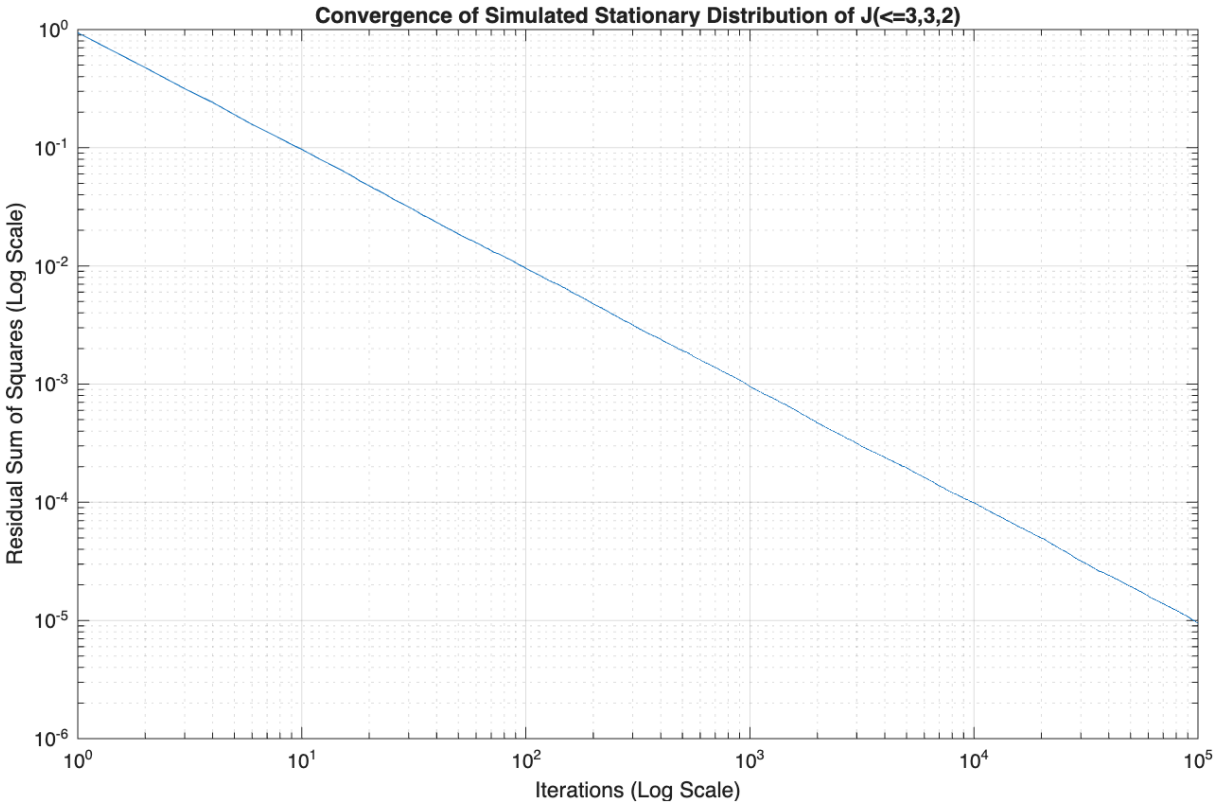


Figure 17: Residual sum of squares plotted against number of iterations to show convergence of simulated stationary distribution with theoretical stationary state for juggling system $J(\leq 3, 3, 2)$ plotted on a log-log plot

The following graph 18 has been referred to in section 6.1. This figure compares the probabilities of

[62] The result is stored on line 70 . The calculation to produce this resulting vector is done on the same line of the same file.

[63] The calculation used to generate this can be found on line 68

[64] The calculation is done on lines 72–75

simulated stationary state (blue) against theoretical stationary state (red) for juggling system $J(\leq 3, 3, 1)$

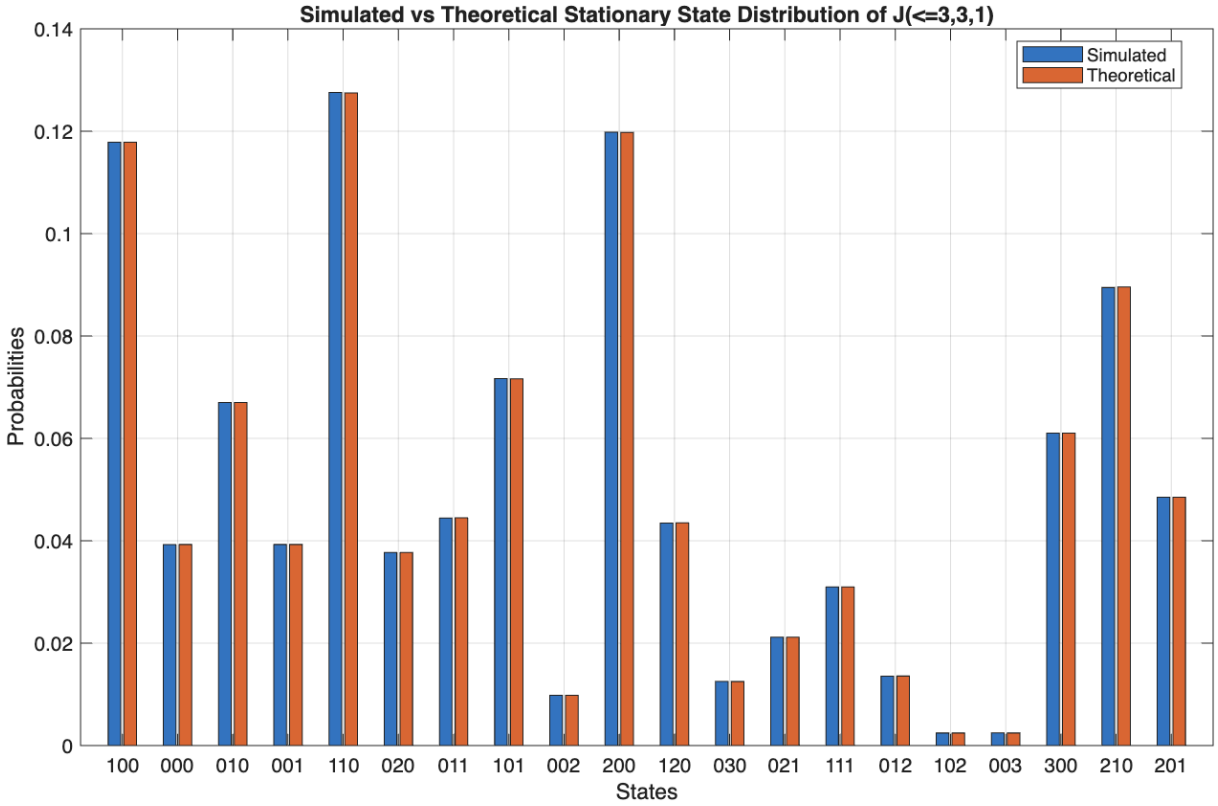


Figure 18: Comparison of the probabilities of simulated stationary state (blue) against theoretical stationary state (red) for juggling system $J(\leq 3, 3, 1)$

The following graph 19 has been referred to in section 6.1. It compares the probabilities of simulated stationary state (blue) against theoretical stationary state (red) for juggling system $J(\leq 3, 3, 2)$.

The following graph 20 has been referred to in section 5.1. This figure describes the residual sum of squares plotted against number of iterations to show convergence of simulated stationary distribution with theoretical stationary state for juggling system $J(\leq 3, 3, 3)$ plotted on a log-log plot.

This graph 21 is mentioned in 6.1. This graph compares the probabilities of simulated stationary state (blue) against theoretical stationary state (red) for juggling system $J(\leq 3, 6, 3)$.

The following graph 22 has been referred to in section 5.1. This graph describes the residual sum of squares plotted against number of iterations to show convergence of simulated stationary distribution with theoretical stationary state for juggling system $J(\leq 3, 6, 1)$ plotted on a log-log plot

This graph 23 is referred in section 6.1. This graph compares the probabilities of simulated stationary state (blue) against theoretical stationary state (red) for juggling system $J(\leq 3, 3, 1)$

This terminal 24 has been produced using calculations on the lines 98–118 of the file `SimRun_1Bay.m`

A.9 Simulation: Results from Terminal

This terminal 25 has been produced using calculations on the lines 102–121 of the file `SimRun_2Bay.m`

This terminal 26 has produced using calculations on the lines 99–118 of the file `SimRun_3Bay.m`

This figure 27 shows run time, economic values and rssq for $J(\leq 3, 6, 1)$. This terminal 27 has produced using calculations on the lines 163–175 of the file `SimRun_1Bay_6H`

This 28 image compares the probabilities of simulated stationary state (blue) against theoretical stationary state (orange) for juggling system $J(\leq 3, 6, 1)$ the simulated and station. This terminal 28 has produced using calculations on the lines 179–182 of the file `SimRun_1Bay_6H`

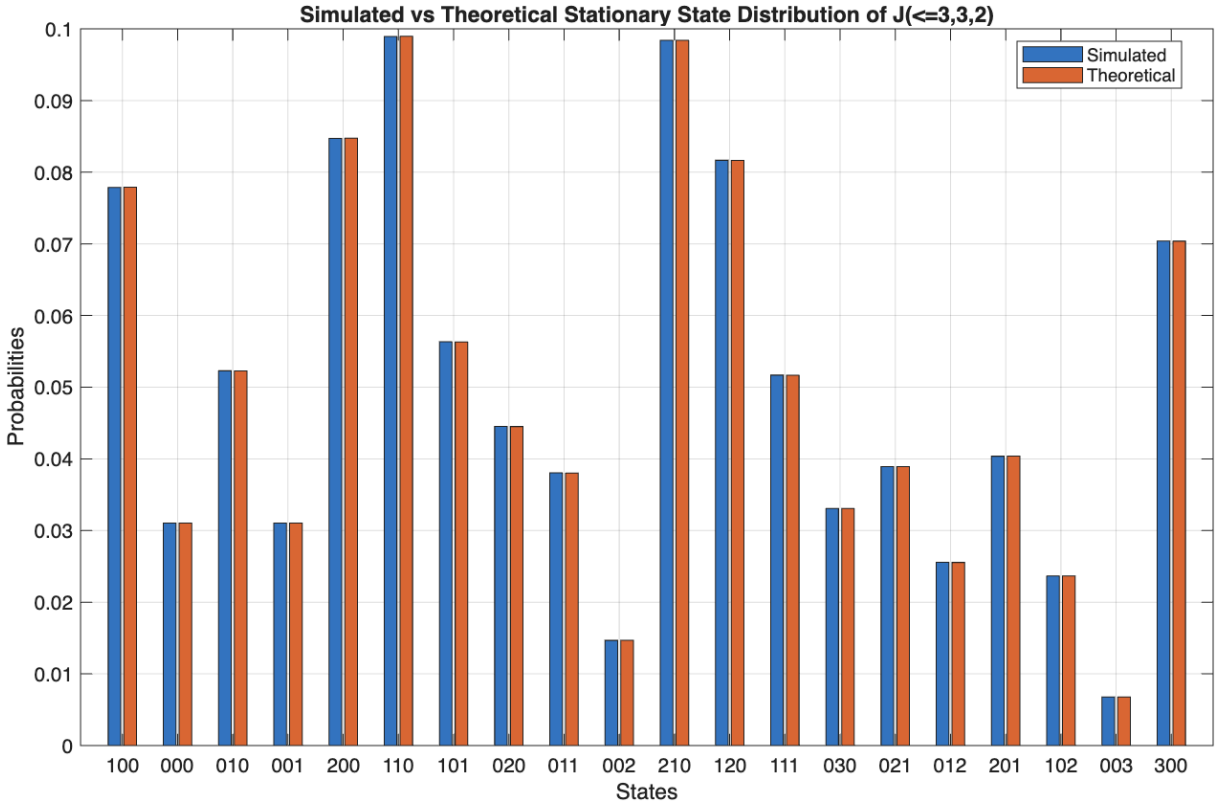


Figure 19: Comparison of the probabilities of simulated stationary state (blue) against theoretical stationary state (red) for juggling system $J(\leq 3, 3, 2)$

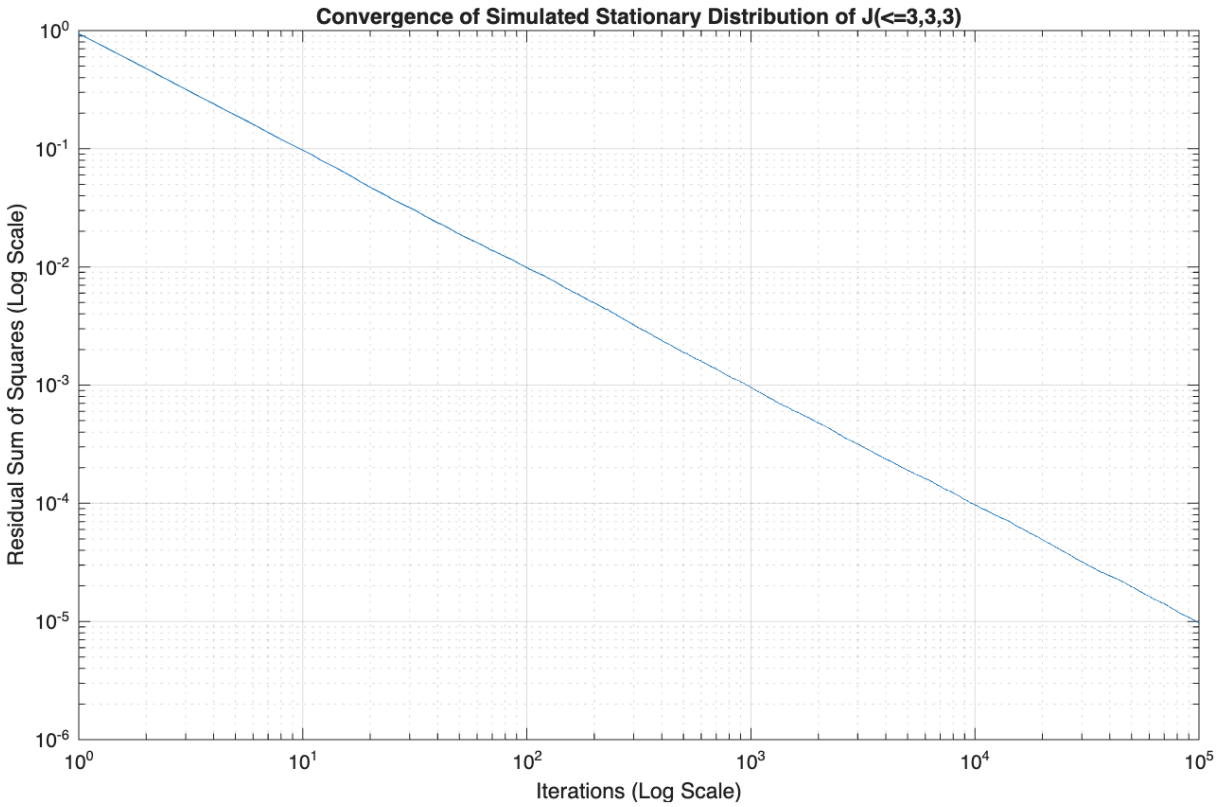


Figure 20: Residual sum of squares plotted against number of iterations to show convergence of simulated stationary distribution with theoretical stationary state for juggling system $J(\leq 3, 3, 3)$ plotted on a log-log plot



Figure 21: Comparison of the probabilities of simulated stationary state (blue) against theoretical stationary state (red) for juggling system $J(\leq 3, 6, 3)$

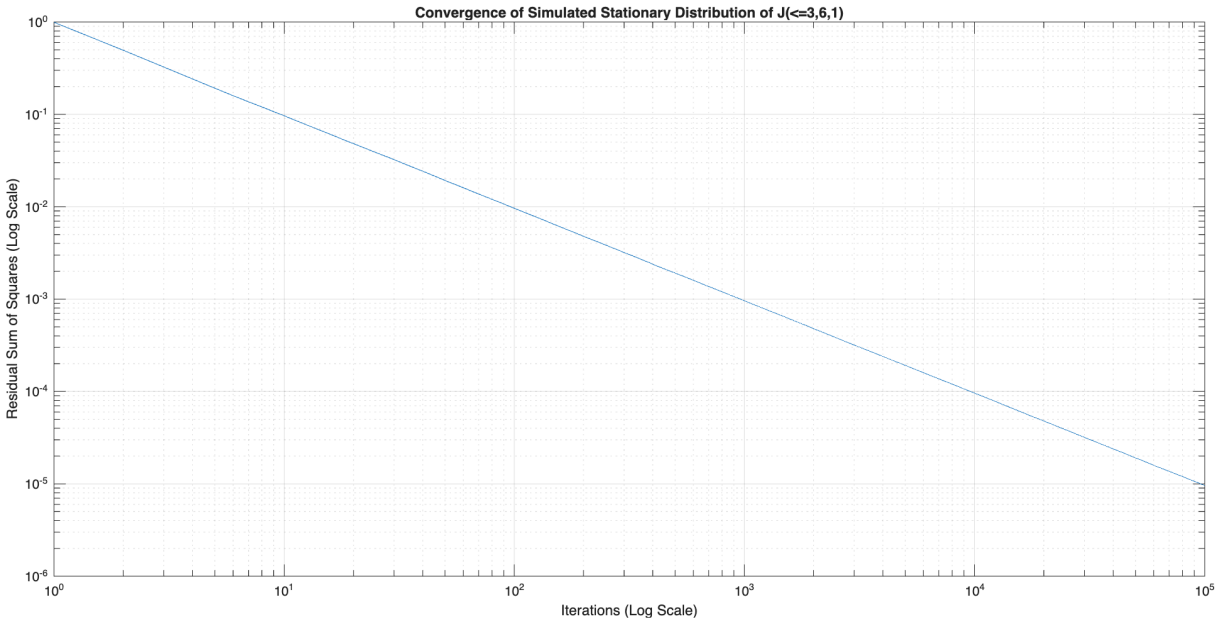


Figure 22: Residual sum of squares plotted against number of iterations to show convergence of simulated stationary distribution with theoretical stationary state for juggling system $J(\leq 3, 6, 1)$ plotted on a log-log plot

A.10 Economic Results

The following table 2 lists the run time, average cost, average revenue and average profit against the various experiments. These values in table 2 have been extracted and collected from output images in the terminal section A.9 above. The values have been extracted from terminal 24, terminal 25, terminal 26 and terminal 27. The values are stored in the file `Cost Results.xlsx`

They have been referenced in section 5.3. The figures 29 and 30 can be found saved in `Cost Results.xlsx`.

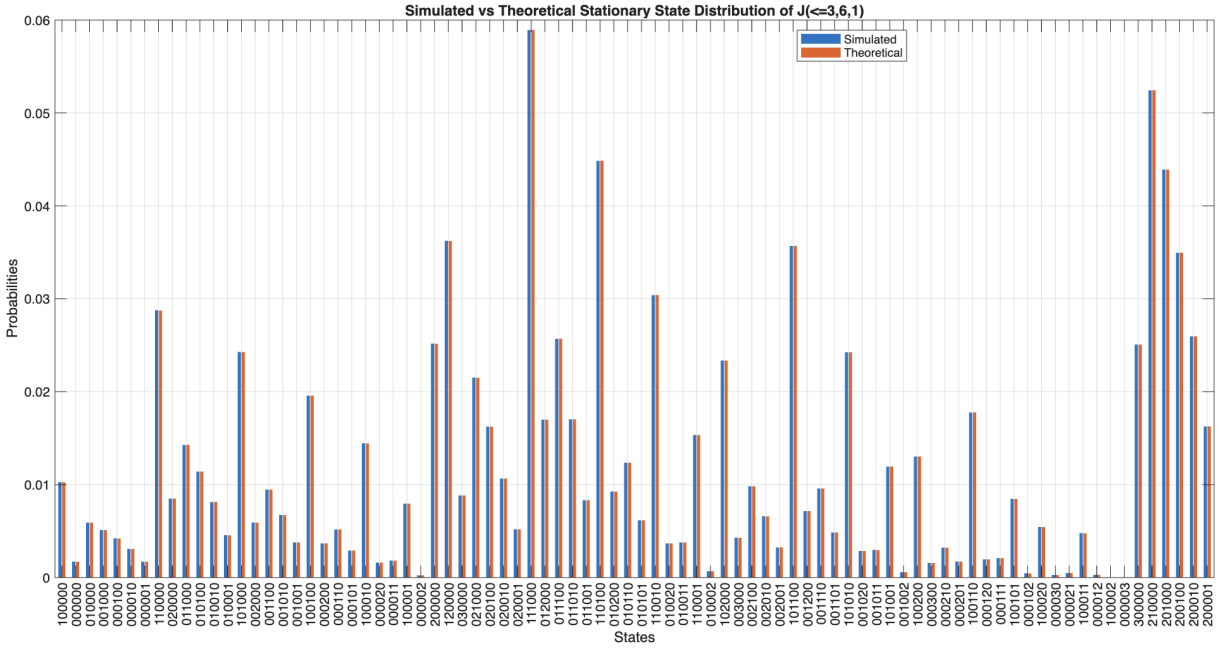


Figure 23: Comparison of the probabilities of simulated stationary state (blue) against theoretical stationary state (red) for juggling system $J(\leq 3, 6, 1)$

```
%%%%%%%%%%%%%%%%%
    "Juggling system: "    " J(<=3,3,1)"

The loop took 60.7714 seconds.
It cost 1224479.7458
Revenue generated 1069165.075
RSSQ is calculated out to be 3.3682e-08
%%%%%%%%%%%%%%%%%
The theroetical stationary vector is as follows
  Columns 1 through 14

    0.1178    0.0393    0.0670    0.0393    0.1275    0.0377    0.0445    0.0716    0.0098    0.1198    0.0435    0.0125    0.0212    0.0310

  Columns 15 through 20

    0.0136    0.0025    0.0025    0.0610    0.0896    0.0485

The simulated stationary vector is as follows
  Columns 1 through 14

    0.1178    0.0393    0.0670    0.0393    0.1276    0.0377    0.0445    0.0717    0.0098    0.1198    0.0434    0.0125    0.0212    0.0310

  Columns 15 through 20

    0.0136    0.0025    0.0025    0.0610    0.0895    0.0485
```

Figure 24: Terminal output showing run time, economic values, rssq and theoretical and simulated stationary state vectors for $J(\leq 3, 3, 1)$

	Exp 1	Exp 2	Exp 3	Exp 4
Run Time	60.01	66.44	63.09	261.73
Avg Cost	1,224,479.75	1,268,639.92	1,493,720.53	911,847.61
Avg Revenue	1,069,165.08	1,466,897.28	1,490,580.80	862,218.91
Avg Profit	(155,314.67)	198,257.35	(3,139.73)	(49,628.70)

Table 2: Comparing the run time, average revenue, average cost and average profit with different number of bays and throw heights.

The first image 29 compares the average revenue and average cost while varying the number of bays and throw-heights. The second image 30 plots the profit while varying the number of bays and throw-heights.

```
%%%%%%%%%%%%%%%%
    "Juggling system: "    " J(<=3,3,2)"

The loop took 64.9835 seconds.
It cost 1268639.9208
Revenue generated 1466897.275
RSSQ is calculated out to be 6.891e-09
%%%%%%%%%%%%%%%%
The theroetical stationary vector is as follows
  Columns 1 through 14

    0.0779    0.0311    0.0523    0.0311    0.0847    0.0990    0.0563    0.0445    0.0380    0.0147    0.0984    0.0817    0.0517    0.0331

  Columns 15 through 20

    0.0389    0.0256    0.0404    0.0237    0.0068    0.0704

The simulated stationary vector is as follows
  Columns 1 through 14

    0.0779    0.0311    0.0523    0.0311    0.0847    0.0989    0.0563    0.0445    0.0381    0.0147    0.0984    0.0817    0.0517    0.0331

  Columns 15 through 20

    0.0389    0.0256    0.0404    0.0237    0.0068    0.0704
```

Figure 25: Terminal output showing run time, economic values, rssq and theoretical and simulated stationary state vectors for $J(\leq 3, 3, 2)$

```
    "Juggling system: "    " J(<=3,3,3)"

The loop took 63.5722 seconds.
It cost 1493720.5333
Revenue generated 1490580.8
RSSQ is calculated out to be 1.1504e-08
%%%%%%%%%%%%%%%%
The theroetical stationary vector is as follows
  Columns 1 through 14

    0.0674    0.0302    0.0493    0.0302    0.0778    0.0884    0.0486    0.0442    0.0374    0.0163    0.0808    0.1017    0.0399    0.0847

  Columns 15 through 20

    0.0494    0.0228    0.0439    0.0436    0.0309    0.0126

The simulated stationary vector is as follows
  Columns 1 through 14

    0.0673    0.0302    0.0494    0.0302    0.0778    0.0883    0.0486    0.0442    0.0374    0.0163    0.0808    0.1017    0.0399    0.0847

  Columns 15 through 20

    0.0493    0.0228    0.0439    0.0436    0.0309    0.0126
```

Figure 26: Terminal output showing run time, economic values, rssq and theoretical and simulated stationary state vectors for $J(\leq 3, 3, 3)$

```
%%%%%%%%%%%%%%%%
    "Juggling system: "    " J(<=3,6,1)"

The loop took 285.1659 seconds.
It cost 911847.6091
Revenue generated 862218.9107
RSSQ is calculated out to be 5.9648e-09
%%%%%%%%%%%%%%%%
```

Figure 27: Terminal output showing run time, economic values, rssq and theoretical and simulated stationary state vectors for $J(\leq 3, 6, 1)$

```
The theroetical stationary vector is as follows
  Columns 1 through 14

    0.0103    0.0017    0.0059    0.0051    0.0042    0.0031    0.0017    0.0287    0.0085    0.0143    0.0114    0.0081    0.0046    0.0243

  Columns 15 through 28

    0.0059    0.0095    0.0067    0.0038    0.0196    0.0037    0.0052    0.0029    0.0144    0.0016    0.0018    0.0079    0.0002    0.0252

  Columns 29 through 42

    0.0362    0.0088    0.0215    0.0162    0.0107    0.0052    0.0590    0.0170    0.0257    0.0170    0.0083    0.0449    0.0093    0.0124

  Columns 43 through 56

    0.0062    0.0304    0.0037    0.0038    0.0153    0.0007    0.0234    0.0043    0.0098    0.0066    0.0032    0.0357    0.0071    0.0096

  Columns 57 through 70

    0.0048    0.0242    0.0029    0.0030    0.0119    0.0006    0.0130    0.0016    0.0032    0.0017    0.0178    0.0020    0.0021    0.0085

  Columns 71 through 84

    0.0005    0.0054    0.0003    0.0005    0.0048    0.0003    0.0000    0.0000    0.0251    0.0525    0.0439    0.0350    0.0260    0.0163

The simulated stationary vector is as follows
  Columns 1 through 14

    0.0103    0.0017    0.0059    0.0051    0.0042    0.0031    0.0017    0.0288    0.0085    0.0143    0.0114    0.0081    0.0046    0.0243

  Columns 15 through 28

    0.0059    0.0095    0.0067    0.0038    0.0196    0.0037    0.0052    0.0029    0.0144    0.0016    0.0018    0.0079    0.0002    0.0252

  Columns 29 through 42

    0.0363    0.0088    0.0215    0.0162    0.0107    0.0052    0.0589    0.0170    0.0257    0.0170    0.0083    0.0448    0.0093    0.0124

  Columns 43 through 56

    0.0062    0.0304    0.0037    0.0038    0.0153    0.0007    0.0234    0.0043    0.0098    0.0066    0.0032    0.0357    0.0071    0.0096

  Columns 57 through 70

    0.0049    0.0242    0.0029    0.0030    0.0119    0.0006    0.0130    0.0016    0.0032    0.0017    0.0178    0.0020    0.0021    0.0085

  Columns 71 through 84

    0.0005    0.0054    0.0003    0.0005    0.0048    0.0003    0.0000    0.0000    0.0251    0.0524    0.0439    0.0350    0.0260    0.0163
```

Figure 28: Comparison of the probabilities of simulated stationary state (blue) against theoretical stationary state (orange) for juggling system $J(\leq 3, 6, 1)$

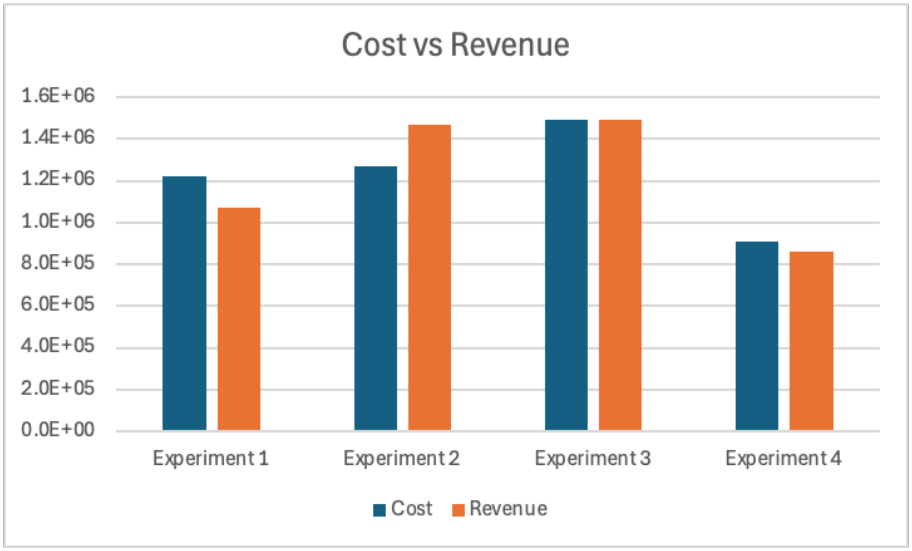


Figure 29: Comparing the average revenue and average cost while varying the number of bays and throw-heights. Source: Table 2



Figure 30: Plotting profit while varying the number of bays and throw-heights. Source: Table 2